

## Assignment-4

Name - Priyanshi Patel

Roll no. - 2401010268

Section - A

Program - B.Tech CSE

Course - Database Management System

Course Code - ENCS204

Submitted to:- Manvi Kagal

# Section :- A :-

Ques:- What is transaction in DBMS?

Ans:- A transaction in DBMS is a logical unit of work that consists of one or more database operations (such as INSERT, UPDATE, DELETE, SELECT). These operations must be executed completely or not at all.

→ A transaction ensures that the database remains consistent even if something goes wrong.

→ Ex:- Bank transfer:-

- Debit ₹500 from Account A
- Credit ₹500 to Account B.

→ Both steps must happen together. If one fails, the whole transaction is rolled back.

→ Key Points:-

→ Begin with BEGIN TRANSACTION

→ Ends with COMMIT (save) or ROLLBACK (undo)

→ Maintains data integrity.

Ques:- 2 Explain ACID properties.

Ans:- ACID properties ensure reliable & safe transaction processing.

→ A → Atomicity (All or Nothing)

- Either all operations execute or none
  - If one fails → entire transaction is rolled back
- Ex:- If money is debited but not credited → rollback.

→ C - Consistency (Valid State)

- Database must move from one valid state to another.
  - Rules, constraints & integrity must be maintained
- Ex:- Account balance cannot be -ve if not allowed.

→ Isolation (No Interference)

- Transaction execute independently.
  - Intermediate states are not visible to other transactions
- Ex:- Two users updating same account should not conflict.

→ Durability :- (Permanent Save)

- Once committed, data is permanently stored
  - Even system crash cannot lose committed data
- Ex:- After payment success, data remains saved.

Ques:- 3 what is concurrency control?

Ans:- Concurrency control is the process of managing multiple transactions executing at the same time without causing data inconsistency.

→ It ensures that results are correct & conflict-free

→ Problems without concurrency control.

- Lost Update
- Dirty Read
- Unrepeatable Read

→ Techniques used:-

- Locking (Shared & Exclusive locks)
- Time stamp ordering

## • Multiversion Concurrency Control (MVCC)

→ Ex:- Two users editing same bank balance simultaneously → system must handle properly.

Ques:- 4 what is a deadlock?

Ans:- A deadlock occurs when two or more transactions are waiting for each other indefinitely & once none can proceed.

→ It creates a cycle of waiting.

Ex:- • Transaction  $T_1$  locks resource A & wait for B.

• Transaction  $T_2$  locks resource B & wait's for A.

→ Both stuck forever

→ Solutions:-

- Deadlock prevention (avoid conditions)
- Deadlock detection (find & resolve)
- Deadlock recovery (rollback one transaction)

5) what is serializability?

Ans:- Serializability ensures that the result of concurrent transactions is the same as if they were executed one after another (serially).

→ It is the highest level of correctness in concurrency control.

→ Types:-

- conflict Serializability
- View Serializability.

→ Example:- If  $T_1$  &  $T_2$  runs together, result should match either:

•  $T_1 \rightarrow T_2$

OR

•  $T_2 \rightarrow T_1$

Ques:- 6 What is recovery in DBMS?

Ans:- Recovery is the process of restoring the database to a correct state after failure (like crash, power failure, system error).

→ Ensures data is not lost.

→ Techniques:-

- Log-based recovery
- Checkpoints
- Backup & restore

→ Ex:- If system crashes after money is debited but before credit → recovery restores correct data state.

Ques:- 7 What is a checkpoint?

Ans:- A checkpoint is a mechanism where the DBMS saves the current state of the database to stable storage.

→ It reduces recovery time.

→ How it works:-

- DBMS writes all change to disk
- Marks a checkpoint in log file
- During recovery → starts from checkpoint instead of beginning.

→ Ex:- Instead of reprocessing all transactions, system restarts from last checkpoint.

## # Section B :-

Ques:- 1 Explain ACID Properties using Banking Example.

Ans:- Consider a bank transfer of ₹1000 from Account A to Account B.

## \* Atomicity (All or nothing)

- Either both operations happen:
  - Debit from A
  - Credit to B

- Or none happen

→ If system crashes after debiting A but before crediting B → transaction is rolled back.

## # Consistency (Valid State)

- Before transaction: Total money =  $A + B$
- After transaction: Total money should remain same
- If A had £5000 & B had £2000 → total £7000
- After transaction → still £7000 (no data corruption)

## # Isolation:- (No Interference)

- If two users access same account simultaneously.
- one transaction should not see intermediate state
- User 1 transferring money & User 2 checking balance → User 2 should not see half-updated balance.

## # Durability (Permanent Storage):

- Once transaction is committed:
  - Data is permanently saved.
- Even if system crashes after successful transfer → updated balances remain safe/secure.

Ques:- 2 Illustrate a Deadlock Situation with Two Transactions

Ans:- Two Transactions  $T_1$  &  $T_2$  & two resources: Account A & Account B

→ Execution:-  $T_1$  locks Account A

•  $T_2$  locks Account B

Now:-

•  $T_2$  wants to lock B → waits for  $T_1$

•  $T_1$  wants to lock A → waits for  $T_2$

→ Both are waiting for each other → Deadlock occurs

$T_1$  → holds A, waiting for B

$T_2$  → holds B, waiting for A

# locks are used to prevent conflicts

• shared lock (Read)

• Exclusive Lock (Write)

→ Deadlock happens due to improper locking order.

# How to handle Deadlock:-

• Prevention (avoid circular wait)

• Detection (wait for graph)

• Recovery (rollback one transaction)

Ques:- 3 Explain Serial vs Concurrent Execution with Example.

Ans:- Serial Execution:- Transactions execute one after another.

Ex:-  $T_1$ : Read A write A

$T_2$ : Read A write A

→ first  $T_1$  complete, then  $T_2$  starts

→ No conflicts

→ Easy to manage

→ slow performance

# Concurrent Execution :- Transaction execute simultaneously.

Ex:-  $T_1$  : Read A  $\rightarrow$  May cause problems like :  
 $T_2$  : Read A  $\rightarrow$  Lost Update  
 $T_1$  : Write A  $\rightarrow$  Dirty Read  
 $T_2$  : Write A

| Feature     | Serial Execution | Concurrent Execution |
|-------------|------------------|----------------------|
| Speed       | Slow             | Fast                 |
| Safety      | High             | Needs Control        |
| Complexity  | Simple           | Complex              |
| Data Issues | No               | Possible.            |

## # Section : c

Ques: A banking system transfers money b/w accounts. Explain how ACID properties apply.

Ans:- Consider a transaction : Transfer ₹1000 from Account A to Account B.

$\rightarrow$  Atomicity :- • Both operations must occur:  
• Debit ₹1000 from A  
• Credit ₹1000 to B

$\rightarrow$  If one fails  $\rightarrow$  entire transaction is rolled back,  
 $\rightarrow$  Ensures no partial update.

$\rightarrow$  Consistency :-  $\rightarrow$  Total balance must remain constant  
 $\rightarrow$  Database rules (like minimum balance) must be maintained

Ex:- A+B total remains unchanged after transfer.

# Isolation :-  $\rightarrow$  Multiple transaction should not interfere

$\rightarrow$  Intermediate state (like money deducted but not added) is not visible

→ other users see either old or new balance, not half-done transaction

# Durability:- • Once transaction is committed  
→ changes are permanently saved  
→ Even after system crash, updated balances remain safe.

Ques:- 2 Two Transactions update same data simultaneously.  
Identify problem issues.

Ans:- when two transactions run at the same time on same data, concurrency problems may occur:

1) Last Update problem:-

→ one transaction overwrites another's update

ex:- →  $T_1$  reads balance = 1000

→  $T_2$  reads balance = 1000

→  $T_1$  updates → 1200

→  $T_2$  updates → 1500

→ final value = 1500 ( $T_1$  update lost)

2) Dirty Read:-

• One transaction reads uncommitted data

Ex:- •  $T_1$  updates balance but not committed

•  $T_2$  reads that value.

•  $T_2$  rolls back

→  $T_2$  used invalid data.

③ Unrepeatable Read:-

→ Same query gives different results

ex:-  $T_1$  reads balance = 1000

$T_2$  updates balance to 2000

• ? read again  $\rightarrow$  2000

#### ④ Phantom Read (extra point)

- New rows appear during transaction

Ques:- A system crashes during transaction. Explain recovery steps.

Ans:- When a system crashes, DBMS uses recovery mechanisms to restore data.

#### Step ① :- Use log file

- DBMS maintains a log file with all operations:  
 $\rightarrow$  start Transaction  $\rightarrow$  update  $\rightarrow$  Commit / Rollback

#### Step ② : Identify Transaction State

- Committed transactions  $\rightarrow$  must be REDO
- Uncommitted transactions  $\rightarrow$  must be UNDO

#### Step ③ : Perform Recovery :-

REDO (Repeat)

- Reapply committed transactions  $\rightarrow$  Ensures durability.

#### Step ④ : Undo (Rollback)

- Reverse incomplete transactions  $\rightarrow$  Ensures atomicity.

#### Step ⑤ :- Checkpoint (if available)

- $\rightarrow$  Start recovery from last checkpoint
- $\rightarrow$  Reduces recovery time.

#### #Section: D

1) Explain one ACID property with real-life example.

Ans:- Atomicity means a transaction must be fully completed or not executed at all.

$\rightarrow$  Ex:- online shopping Payment

- 1) money deducted
- 2) order is confirmed

- If payment is deducted but order is not placed → system will refund (rollback)
  - If both success → transaction is committed.
- 4) Draw a transaction timeline & identify conflict.
- Ans:- Timeline example:-
- Time →  $T_1$  : Read(A) - - - - - write(A)  
 $T_2$  : Read(A) - - - - - write(A)
- $T_1$  reads A
  - Before  $T_1$  writes,  $T_2$  also reads A
  - Both update A independently
  - Final value may overwrite one transaction
- ③ Explain Recovery technique
- Ans:- → Log-Based Recovery
- DBMS keeps a log file that records every transaction step.
- Log contains:-
- Transaction start
  - Data changes
  - Commit/Rollback
- During Crash:-
- Committed transactions → REDO
  - Uncommitted transactions → UNDO
- Ex:-  $T_1$  : start → update → commit  
 $T_2$  : start → update (crash)
- After recovery :  $T_1$  → REDO  
 $T_2$  → UNDO
- Log-based recovery ensures:-
- Data consistency
  - No partial updates
  - Fast restoration after failure.